GDB ��� A Practical Guide

The commands you actually reach for, and when

- How to use this
- Part 1 — The Essentials
  - 1.1 Getting in
  - 1.2 The five commands that cover most sessions
  - 1.3 A crash — the standard workflow
  - 1.4 Core dumps
  - 1.5 Breakpoints
  - 1.6 Stepping
  - 1.7 Inspecting data
- Part 2 — C++ Specifics
  - 2.1 Printing STL containers
  - 2.2 Useful display settings
  - 2.3 Polymorphism
  - 2.4 Exceptions
  - 2.5 Optimised builds
- Part 3 — Threads
  - 3.1 The commands
  - 3.2 Diagnosing a deadlock
  - 3.3 Controlling thread scheduling
  - 3.4 The honest limitation
- Part 4 — Beyond the Basics
  - 4.1 Reverse debugging
  - 4.2 Changing state at runtime
  - 4.3 Scripting
  - 4.4 `~/.gdbinit`
  - 4.5 Attaching to a live process
- Part 5 — The Wider Toolbox
  - Related binary tools
- Part 6 — Quick Reference
- If you’re asked about gdb in an interview

How to use this

Organised by **what’s gone wrong**, not by command category — because that’s how you’ll need it. The commands you’ll use 90% of the time are in Part 1; the rest is for when the easy path fails.

Every command has a short form. Use them: `b` for `break`, `r` for `run`, `c` for `continue`, `n` for `next`, `s` for `step`, `p` for `print`, `bt` for `backtrace`.

Part 1 — The Essentials

1.1 Getting in

```
gdb ./myprogram                 # start under gdb
gdb --args ./myprogram -v foo   # with command-line arguments
gdb ./myprogram core            # post-mortem, from a core file
gdb -p 12345                    # attach to a RUNNING process
gdb ./myprogram -ex run -ex bt  # run and backtrace automatically, then stop
```

Compile with `-g`, always, even in release builds:

```
g++ -O2 -g myprog.cpp -o myprog
```

`-g` adds debug symbols and **does not slow the program down** — it only makes the binary larger. Shipping without it means a production crash gives you hex addresses instead of line numbers.

For heavy debugging use `-O0 -g` so variables aren't optimised away. At `-O2` you'll frequently see `<optimized out>`, and the line numbers will jump around because of inlining and reordering.

1.2 The five commands that cover most sessions

```
run          (r)   start the program
backtrace    (bt)  show the call stack       <-- the single most useful command
print x      (p)   show a variable
break f      (b)   stop at function f
continue     (c)   carry on
```

If you learn nothing else, learn `run`, `bt`, and `print`.

1.3 A crash — the standard workflow

```
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x... in Order::price (this=0x0) at order.cpp:42

(gdb) bt                    # where am I, and how did I get here?
#0  Order::price() at order.cpp:42
#1  Book::bestBid() at book.cpp:88
#2  Strategy::onTick() at strategy.cpp:15
#3  main() at main.cpp:30

(gdb) frame 1               # move to the caller's frame
(gdb) list                  # show the source around here
(gdb) info locals           # all local variables in this frame
(gdb) print order           # inspect the suspicious one
$1 = (Order *) 0x0          # there it is -- null pointer
```

That sequence — `bt`, `frame`, `info locals`, `print` — resolves most crashes.

`bt full` shows locals for every frame at once. Noisy, but sometimes faster.

1.4 Core dumps

A core dump is a snapshot of the process at the moment it died. Essential for production crashes you can't reproduce.

```
ulimit -c unlimited                     # enable core dumps (per shell)
cat /proc/sys/kernel/core_pattern       # where they go
gdb ./myprogram core.12345              # open it
(gdb) bt                                # same workflow as above
```

On systemd systems: `coredumpctl list`, then `coredumpctl gdb <pid>`.

The one gotcha: the binary must match the core exactly. A rebuild between the crash and the debug session gives you nonsense stack traces.

1.5 Breakpoints

```
break main                    # by function
break order.cpp:42            # by file and line
break Book::insert            # by method
break order.cpp:42 if id == 7 # CONDITIONAL -- very useful
tbreak main                   # temporary: deleted after first hit

info breakpoints              # list them
delete 2                      # remove breakpoint 2
disable 2 / enable 2          # toggle without deleting
```

Conditional breakpoints are the ones that save real time. Stopping on every one of a million iterations is useless; stopping only when the condition is wrong is not:

```
(gdb) break process.cpp:88 if qty < 0
(gdb) break book.cpp:120 if strcmp(symbol, "RELIANCE") == 0
```

Watchpoints stop when a *value* changes, rather than when a line is reached:

```
watch counter                 # break when counter changes
watch -l ptr->field           # watch the location, not the expression
rwatch x                      # break on READ
```

Invaluable for "who is corrupting this variable?" — but hardware watchpoints are limited (usually 4) and software ones are extremely slow.

1.6 Stepping

```
next    (n)    execute the next line, stepping OVER function calls
step    (s)    execute the next line, stepping INTO calls
finish  (fin)  run until the current function returns, and show the return value
until          continue until a line past the current one (escapes loops)
continue (c)   run until the next breakpoint
advance 88     run until line 88 is reached
```

`finish` **is underused.** When you've stepped into something you didn't mean to, it gets you out *and* prints the return value.

`until` **escapes a loop** without setting a breakpoint after it.

1.7 Inspecting data

```
print x             (p)   print a variable
print *ptr                 dereference
print arr[3]               index
print obj.field            member
print/x val                in hex   (/d decimal, /t binary, /c char, /f float)
print sizeof(Order)        expressions work

info locals                all locals in this frame
info args                  function arguments
info registers             CPU registers
display x                  print x automatically at every stop
undisplay 1                stop doing that
```

Printing an array or buffer:

```
p *array@10          # 10 elements starting at array
p arr[0]@5           # same idea
x/16xb ptr           # examine: 16 items, heX, Bytes
x/8xg ptr            # 8 giant (8-byte) words in hex
x/s charptr          # as a string
x/20i $pc            # 20 instructions from the program counter
```

The `x` (examine) format is `x/NFU`: **N**umber, **F**ormat, **U**nit.

Calling functions from the prompt — extremely useful, and easy to forget:

```
(gdb) p myVector.size()
(gdb) p myMap.count("RELIANCE")
(gdb) call dumpState()
```

Careful: this actually executes the function in the target process, side effects included.

Part 2 — C++ Specifics

2.1 Printing STL containers

Modern GDB ships Python pretty-printers for libstdc++, so this usually just works:

```
(gdb) p myVector
$1 = std::vector of length 3, capacity 4 = {10, 20, 30}

(gdb) p myMap
$2 = std::map with 2 elements = {["RELIANCE"] = 100, ["TCS"] = 50}
```

If you instead see raw internals (`_M_start`, `_M_finish`), the printers aren't loaded:

```
(gdb) info pretty-printer        # check what's registered
(gdb) set print pretty on        # nicer struct formatting regardless
```

To enable manually, add to `~/.gdbinit`:

```
python
import sys
sys.path.insert(0, '/usr/share/gcc/python')
from libstdcxx.v6.printers import register_libstdcxx_printers
register_libstdcxx_printers(None)
end
```

Without printers, a vector's contents are between the two pointers:

```
(gdb) p *v._M_impl._M_start@(v._M_impl._M_finish - v._M_impl._M_start)
```

2.2 Useful display settings

```
set print pretty on          # multi-line struct output
set print object on          # show the DYNAMIC type of polymorphic objects
set print vtbl on            # show vtables
set print elements 200       # don't truncate long arrays (0 = unlimited)
set print null-stop on       # stop char arrays at the NUL
set pagination off           # no "---Type <return>---" prompts
```

Put the ones you always want in `~/.gdbinit`.

2.3 Polymorphism

```
(gdb) set print object on
(gdb) p *basePtr
$1 = (DerivedStrategy) {<BaseStrategy> = {...}, extraField = 42}
```

Without `set print object on` you only see the static type, which hides the actual object.

```
(gdb) info vtbl basePtr      # show the vtable -- confirms the dynamic type
```

2.4 Exceptions

```
catch throw                  # break when ANY exception is thrown
catch throw std::runtime_error
catch catch                  # break when one is caught
```

`catch throw` is the right tool for "something throws and I don't know where" — it stops you at the throw site with the stack intact, rather than after unwinding has destroyed it.

2.5 Optimised builds

At `-O2` expect:

- `<optimized out>` for variables kept only in registers
- Line numbers jumping backwards and forwards (instruction reordering)
- Missing stack frames (inlining)

Mitigations:

```
        g++ -O2 -g -fno-omit-frame-pointer   # keeps backtraces reliable
        g++ -Og -g                           # optimise for debugging: fast-ish, still inspectable
```

`-Og` is the right level when you need both speed and a usable debugger. And `-fno-omit-frame-pointer` costs almost nothing while making `perf` and `gdb` stacks far more reliable — worth having in production builds.

Part 3 — Threads

This is where gdb earns its keep, and where an HFT interviewer is most likely to probe.

3.1 The commands

```
info threads                 # list all threads; * marks the current one
thread 3                     # switch to thread 3
thread apply all bt          # BACKTRACE FOR EVERY THREAD   <-- the deadlock command
thread apply all bt full     # ...with locals
```

`thread apply all bt` is the single most valuable command for a hung process. Attach to the stuck process, run it, and read what each thread is waiting on.

3.2 Diagnosing a deadlock

```
        gdb -p <pid>                # attach to the hung process
        (gdb) thread apply all bt
```

Look for threads sitting in `pthread_mutex_lock` or `__lll_lock_wait`. Then work out **which** mutex each is waiting for and which thread holds it. Classic pattern:

```
Thread 1: holds mutex A, waiting for mutex B
Thread 2: holds mutex B, waiting for mutex A
```

Lock-ordering inversion. The fix is a global lock ordering, or not holding two at once.

3.3 Controlling thread scheduling

```
set scheduler-locking on     # only the current thread runs while stepping
set scheduler-locking step   # lock only during step/next  (a good default)
set scheduler-locking off    # default: all threads run
```

Why this matters: without it, stepping through one thread lets the others run freely, so the state changes underneath you and the bug moves. `step` mode is usually the right setting.

```
break foo thread 3           # breakpoint that only fires on thread 3
```

3.4 The honest limitation

gdb is a poor tool for finding data races. Stopping a thread changes the timing, so the race usually stops reproducing — the observer effect in its purest form.

For races, use **ThreadSanitizer** (`-fsanitize=thread`), which detects them on any executed path whether or not they manifest. Use gdb *after* TSan has told you where to look.

Say this if asked. Knowing which tool is wrong for a job is as useful as knowing the commands.

Part 4 — Beyond the Basics

4.1 Reverse debugging

```
record                       # start recording execution
reverse-continue     (rc)    run BACKWARDS to the previous breakpoint
reverse-step         (rs)    step backwards
reverse-next
```

Genuinely magical for “how did this variable become garbage?” — you run backwards to find out.

Cost: very slow, and doesn’t support all instructions. Use on a small reproduction, not a whole session.

`rr` (from Mozilla) is the production-grade version — records once, then lets you replay deterministically forwards and backwards as many times as you like.

4.2 Changing state at runtime

```
set var x = 42               # modify a variable
set var ptr->field = 0
jump 88                      # jump to line 88 (dangerous, skips code)
return 0                     # force the current function to return
```

Useful for testing an error path you can’t easily trigger: set the flag, continue, watch what happens.

4.3 Scripting

```
define bt-all
  thread apply all bt
end

# breakpoint that logs and continues without stopping
break process.cpp:88
commands
  silent
  printf "qty=%lld price=%f\n", qty, price
  continue
end
```

`commands` with `continue` turns a breakpoint into a **printf without recompiling** — which is exactly what you want when the build takes ten minutes.

Batch mode, for scripted or automated debugging:

```
gdb -batch -ex run -ex bt --args ./myprog arg1
```

4.4 `~/.gdbinit`

Worth setting up once:

```
set print pretty on
set print object on
set print elements 0
set pagination off
set history save on
set confirm off
```

4.5 Attaching to a live process

```
gdb -p 12345
(gdb) thread apply all bt      # what's it doing?
(gdb) detach                   # leave it running
```

Attaching stops the process. On a live trading system that's a decision, not a debugging step — you've just stopped quoting. `detach` resumes it.

If attach fails with a permissions error:

```
sudo sysctl kernel.yama.ptrace_scope=0     # allow non-root attach (dev machines only)
```

Part 5 — The Wider Toolbox

Know when gdb is the wrong tool. This is the more useful half of the answer.

Problem	Tool	Why
Segfault, null deref	**gdb** or **ASan**	ASan tells you *which* allocation and where it was freed
Memory leak	**Valgrind** or **LeakSanitizer**	gdb won't find leaks at all
Use-after-free	**AddressSanitizer**	reports both the free site and the use site
Data race	**ThreadSanitizer**	gdb perturbs the timing and hides the bug
Undefined behaviour	**UBSan**	signed overflow, bad shifts, bad casts
Deadlock / hang	**gdb** (`thread apply all bt`)	exactly what it's for
"Why is this slow?"	**perf**	gdb tells you nothing about performance
Which syscalls?	**strace**	
Which library calls?	**ltrace**	
Crash in production	**core dump + gdb**	

```
g++ -fsanitize=address,undefined -g prog.cpp    # ASan + UBSan together
g++ -fsanitize=thread -g prog.cpp               # TSan separately (incompatible with ASan)
valgrind --leak-check=full ./prog
```

Sanitizers are compile-time and much faster than Valgrind (~2× versus ~20×). Valgrind needs no recompilation, which sometimes matters.

Related binary tools

```
addr2line -e ./prog 0x401234    # address -> file:line, for a stack trace without gdb
nm -C ./prog                    # symbols, demangled
objdump -d -C ./prog            # disassemble
c++filt _Z3fooi                 # demangle a C++ symbol
ldd ./prog                      # shared library dependencies
readelf -h ./prog               # ELF header
```

`addr2line` is the one to remember — it turns a bare address from a log into a line number without opening gdb at all.

Part 6 — Quick Reference

Command	Short	Does
`run`	`r`	start
`backtrace`	`bt`	call stack
`backtrace full`	`bt full`	stack + locals
`break f`	`b f`	breakpoint
`break f if cond`		conditional breakpoint
`tbreak f`		one-shot breakpoint
`watch x`		break when x changes
`continue`	`c`	resume
`next`	`n`	step over
`step`	`s`	step into
`finish`	`fin`	run to return
`until`	`u`	escape a loop
`print x`	`p x`	show value
`print/x x`		in hex
`p *arr@10`		10 array elements
`x/16xb p`		examine memory
`info locals`		all locals
`info args`		arguments
`info threads`		list threads
`thread apply all bt`		**all thread stacks**
`frame N`	`f N`	select stack frame
`up` / `down`		move a frame
`list`	`l`	show source
`set var x = 5`		modify a variable
`display x`		auto-print at each stop
`catch throw`		break on exception
`detach`		resume and leave
`quit`	`q`	exit

If you’re asked about gdb in an interview

The honest, useful answer isn’t a command list — it’s a **workflow**:

> *“For a crash I’d get a core dump, open it with the matching binary, and start with `bt` to see the stack, then `frame` and `info locals` to inspect the caller. For a hang I’d attach to the live process and run `thread apply all bt` to see what every thread is waiting on — that finds lock-order inversions immediately.*
>
> *For a data race I wouldn’t use gdb at all — stopping threads changes the timing and the bug stops reproducing. That’s ThreadSanitizer’s job, and gdb comes after it’s told me where to look. Same for leaks and use-after-free: ASan gives you the allocation and free sites, which gdb won’t.*
>
> *The setting that saves the most time is conditional breakpoints — stopping on every iteration of a million-iteration loop is useless; stopping only when the quantity goes negative is not.”*

Knowing which tool is wrong for a job is worth as much as knowing the commands.